

SEABORN

Data Visualization using Python

Complete beginner-friendly study notes

Goal

Understand what each Seaborn graph is used for, how its main parameters work, and when to choose it during EDA.

Learning path

Pandas/DataFrame → Understand variables → Choose a graph → Visualize → Interpret → EDA

1. What is Seaborn?

Seaborn is a Python library used for statistical data visualization. It is built on top of Matplotlib and works very well with Pandas DataFrames.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

Library	Main job
Pandas	Work with and manipulate tabular data
NumPy	Numerical operations and arrays
Matplotlib	Core plotting/visualization library
Seaborn	High-level statistical visualizations

Why visualize data?

Numbers alone can hide patterns. A graph can reveal trends, relationships, distributions, differences between groups, and possible outliers much faster.

2. Univariate, Bivariate & Multivariate

Type	Meaning	Example
Univariate	One variable	Distribution of salary
Bivariate	Two variables	Age vs salary
Multivariate	More than two variables	Petal length vs width, grouped by species

Key idea: Choose the graph based on the question you want the data to answer, not simply because you remember a function name.

3. Core Seaborn Graphs

Function	Main purpose	Typical use
scatterplot()	Relationship between two numerical variables	Age vs salary
lineplot()	Trend/progression	Sales over time
countplot()	Count of categorical observations	Students per department
barplot()	Aggregated numerical value by category	Average salary by department
histplot()	Distribution of a numerical variable	Distribution of age
boxplot()	Distribution, median, spread, outliers	Salary by department
violinplot()	Distribution/density by category	Petal length by species
pairplot()	Relationships among several numerical variables	Quick EDA
heatmap()	Matrix/correlation visualization	Correlation matrix

4. Scatterplot()

Use a scatter plot to examine the relationship between two numerical variables. Each observation is shown as a point.

```
sns.scatterplot(x='sepal_length', y='sepal_width', data=df)
plt.show()
```

Think: “Are these two numerical variables related?”

Using hue

```
sns.scatterplot(
    x='petal_length',
    y='petal_width',
    hue='species',
    data=df
)
plt.show()
```

hue tells Seaborn to use different colors for different categories. It is extremely useful for comparing groups.

5. Lineplot()

A line plot connects observations and is especially useful for showing an ordered trend or progression, such as sales across months.

```
sns.lineplot(x='month', y='sales', data=df, errorbar=None)
plt.show()
```

Your Iris example:

```
sns.lineplot(
    x='sepal_length',
    y='sepal_width',
    data=df,
    errorbar=None
)
plt.show()
```

Important: A line plot with Iris observations can look jagged because the observations are connected. For simply examining the relationship between two measurements, a scatterplot is often more intuitive.

`errorbar=None` removes the uncertainty/error band. A confidence interval can be requested with an appropriate `errorbar` setting.

6. countplot()

Use `countplot()` when you want to see how many observations belong to each category.

```
sns.countplot(data=df, x='species')
plt.show()
```

For the Iris dataset, `species` contains categories such as `setosa`, `versicolor`, and `virginica`. `countplot()` displays their observation counts as bars.

7. barplot()

`barplot()` is generally used when you want a numerical aggregate for each category. By default, Seaborn uses the mean.

```
sns.barplot(data=df, x='department', y='salary')
plt.show()
```

Difference: `countplot` = “How many?” | `barplot` = “What is the average/aggregate value?”

8. histplot()

A histogram shows the distribution of a numerical variable by grouping values into bins.

```
sns.histplot(data=df, x='sepal_length')
plt.show()
```

It helps you inspect where values are concentrated, how spread out they are, and whether the distribution looks skewed or unusual.

9. boxplot()

A boxplot summarizes a numerical distribution and is especially useful for comparing groups and spotting potential outliers.

```
sns.boxplot(data=df, x='sepal_length')
plt.show()
```

Part	Meaning
Median	Middle value of the ordered data
Q1	25th percentile
Q3	75th percentile
IQR	$Q3 - Q1$
Points beyond whiskers	Potential outliers (depending on the rule used)

```
sns.boxplot(data=df, x='species', y='petal_length')
plt.show()
```

This compares the distribution of petal length across species.

10. violinplot()

A violin plot shows the distribution/density of a numerical variable for one or more categories. It can reveal the shape of the distribution more clearly than a simple boxplot.

```
sns.violinplot(data=df, x='species', y='petal_length')
plt.show()
```

11. pairplot()

pairplot() creates a grid of plots comparing several numerical variables. It is useful for quickly exploring relationships in a new dataset.

```
sns.pairplot(df)
plt.show()
```

With categories:

```
sns.pairplot(df, hue='species')
```

EDA use: pairplot() can help you quickly identify possible relationships, clusters, and variables worth investigating further.

12. heatmap()

A heatmap represents values in a matrix using visual intensity. A common Data Science use is visualizing a correlation matrix.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.show()
```

annot=True writes the numerical values inside the cells.

Correlation values are commonly interpreted as:

Correlation	Simple interpretation
Close to +1	Strong positive relationship
Close to 0	Weak/limited linear relationship
Close to -1	Strong negative relationship

Important: Correlation does not automatically mean causation.

13. Important Seaborn Parameters

Parameter	Meaning
x=	Column used on the X-axis
y=	Column used on the Y-axis
data=	DataFrame containing the columns
hue=	Color/group according to a variable
style=	Use marker/line style according to a variable
size=	Represent a variable through visual size
errorbar=	Control uncertainty/error representation
annot=True	Write values inside a heatmap

14. How to choose the right graph

Question	Recommended graph
How many observations are in each category?	countplot()
How are two numerical variables related?	scatterplot()
How does a value change over an ordered sequence/time?	lineplot()
What is the distribution of a numerical variable?	histplot()
Are there potential outliers?	boxplot()
How does a numerical variable differ across categories?	boxplot() / violinplot()
What are the relationships among many numerical variables?	pairplot()
Which numerical variables are correlated?	heatmap()

15. Univariate vs Bivariate vs Multivariate

Type	Example code idea	Question
Univariate	histplot(x='salary')	What is the salary distribution?
Bivariate	scatterplot(x='age', y='salary')	Are age and salary related?
Multivariate	scatterplot(..., hue='department')	Does the relationship differ by department?

16. EDA Workflow using Seaborn

When you receive a new dataset, don't immediately start making random graphs. Follow a logical process.

Step	What to do
1. Understand	head(), shape, columns, dtypes, info()
2. Classify	Identify numerical and categorical variables
3. Distributions	Use histplot(), boxplot(), etc.
4. Categories	Use countplot() and category-based plots
5. Relationships	Use scatterplot(), lineplot(), pairplot()
6. Correlation	Use corr() + heatmap()
7. Investigate	Look for outliers, patterns, missing values, unusual behavior
8. Interpret	Write what the graphs actually tell you

17. Iris Dataset Practice

Use this dataset to practice the complete Seaborn workflow:

```
df = sns.load_dataset('iris')

print(df.head())
print(df.shape)
print(df.dtypes)
print(df['species'].unique())
print(df['species'].value_counts())
```

Then practice these visualizations:

```
sns.scatterplot(data=df, x='sepal_length', y='sepal_width')

sns.scatterplot(data=df, x='petal_length', y='petal_width', hue='species')

sns.histplot(data=df, x='petal_length')

sns.boxplot(data=df, x='species', y='petal_length')

sns.pairplot(df, hue='species')

sns.heatmap(df.corr(numeric_only=True), annot=True)

plt.show()
```

18. Quick Revision Sheet

Memorize the purpose first. Syntax comes second.

Function	Remember it as...
scatterplot()	Relationship between two numerical variables
lineplot()	Trend/progression
countplot()	Count categories
barplot()	Aggregate numerical value by category
histplot()	Distribution
boxplot()	Median + spread + possible outliers
violinplot()	Distribution/density by category
pairplot()	Many pairwise relationships
heatmap()	Matrix/correlation visualization

19. Final Mental Map

Seaborn

|
├─ Relationship → scatterplot(), lineplot()
├─ Categories → countplot(), barplot()
├─ Distribution → histplot(), boxplot(), violinplot()
└─ EDA / Multiple variables → pairplot(), heatmap()

Best learning method: write each graph yourself, change the columns, observe the result, and explain the graph in your own words. That is how Seaborn becomes a Data Science skill rather than a collection of commands.

20. Practice Questions

- What is the difference between univariate and bivariate visualization?
- What is the difference between countplot() and barplot()?
- Why would you use hue='species'?
- When is scatterplot() more appropriate than lineplot()?
- What information does a boxplot provide?
- What does errorbar=None do?
- What does annot=True do in heatmap()?
- Why is pairplot() useful during EDA?